# /backend

Execution backend surface: /backend [] | /backend show | /backend help

## Overview

`/backend` is the TUI surface for the execution backend (spec `openhands` FR-008). An execution backend selects **where** a session's tool invocations actually run: `local` runs tools on the host exactly as before (the default), `docker`/`podman` run them inside a sandbox container, and `remote` drives a second ragent server over its REST+SSE API.

With no argument the command opens an interactive **switcher panel** that shows the active backend and the health of *every* registered backend. Each row carries the backend's id, kind, health (`ok` / `unavailable` / `unknown`), and a secret-free connection summary. The panel is keyboard-driven: `Up`/`Down` move the block cursor, `Enter` switches the active backend to the highlighted row, and `Esc` closes the panel.

The panel is a modal overlay: while it is open it swallows every keystroke and the message input field stays locked.

## Syntax

```
/backend                # open the switcher panel
/backend show           # alias of /backend
/backend list           # alias of /backend
/backend local          # switch to the local host backend
/backend docker         # switch to a Docker sandbox backend
/backend podman         # switch to a Podman sandbox backend
/backend remote         # switch to a remote backend
/backend <id>           # switch to a backend by its registered id
/backend help           # usage help
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/backend` | Open the switcher panel listing the active backend and every registered backend's health |
| `/backend show` / `/backend list` | Alias of `/backend` |
| `/backend <kind>` | Switch directly to a backend kind (`local`, `docker`, `podman`, `remote`) |
| `/backend <id>` | Switch directly to a backend by its registered id (from `backends` in `ragent.json`) |
| `/backend help` | Print the usage table (no state change) |

## Panel keys

| Key | Action |
|---|---|
| `Up` / `Down` | Move the block cursor between rows |

| Key | Action |
|---|---|
| `Enter` | Switch the active backend to the highlighted row |
| `Esc` | Close the panel |

A row whose health is not `ok` is dimmed and **cannot** be selected: the switch is refused and the active backend is left unchanged, so the switcher can never leave the session on a backend that cannot run a tool.

## Health

| Kind | `ok` when | `unavailable` when |
|---|---|---|
| `local` | always (the host is always available) | never |
| `docker` / `podman` | the runtime binary resolves on `PATH` (FR-026) | the binary is missing (the detail names it) |
| `remote` | a base URL and a key are configured (FR-030) | either is absent |

Health is a snapshot taken when the panel opens; the panel does not re-probe on each frame.

## Persistence

A successful switch writes `execution_backend: "<kind>"` into the loaded config file (the project-local `.ragent/ragent.json` when one was loaded, otherwise the global config). The per-turn config cache is invalidated so the next turn resolves and dispatches through the new backend.

A descriptor-free kind switch (`/backend docker`) writes the bare label. To carry an image, workspace, URL, or key, declare a full descriptor in the `backends` array (or as an inline `execution_backend` object) and select it with `/backend <id>`.

## Examples

Open the switcher:

```
/backend
```

Switch to a declared Docker sandbox by id:

```
/backend docker-box
```

Return to the host:

```
/backend local
```

## Output

- The panel renders a bordered modal titled `Execution backend - active: <kind>`; the active row carries an `[active]` marker.
- A switch appends an `From: /backend` message bubble and sets the status to `backend: <kind>`.

- A refusal appends a `[warn]` message explaining why and sets the status to `backend: unavailable` / `backend: unknown`.
- No permission prompt is raised and no LLM call is made.

## Related

- `/backend help` - usage table
- `docs/howtos/sandbox-backends.md` - full guide to the switchable backends, the sandbox mechanism, and a worked Docker/Podman example
- `specs/openhands/SPEC.md` - the execution backend design (`local`/`docker`/`podman`/`remote`)
- `docs/howtos/config.md` §7.36 - the `execution_backend` and `backends` config keys